EduAgent : agent d'admission IA en enseignement supérieur

Documentation du Jour 23 : gouvernance, Agent Card, versionnement SemVer, runbook d'incident et déploiement.

Version active : 1.0.0

KAMAL EL AFOUI

1. Objectif

Donner une identité, une traçabilité et une procédure d'urgence à l'agent d'admission. Le scoring est déterministe et reproductible ; toute décision finale reste humaine.

2. Architecture

L'API FastAPI sert le tableau de bord sur le port 8001. Le pipeline valide, score et route chaque dossier vers une sortie automatique (Risque faible) ou une escalade humaine (Risque élevé). La gouvernance (kill switch, versions, approbation, rollback) est pilotée par la CLI et par l'API protégée par jeton. Le schéma se trouve dans docs/architecture.svg.

Déploiement : docker compose up --build (service eduagent-api, port 8001, volumes outputs/logs/governance).

3. Workflow d'approbation

1. Proposition (CR-xxx, statut en_revue)
2. Revue technique (responsable technique)
3. Validation pédagogique (responsable pédagogique)
4. Contrôle de non-régression sur les cas golden
5. Déploiement SemVer avec trace d'audit et tag Git
6. Surveillance et rollback selon le runbook

Workflows Langflow fournis dans workflow/ :

AI_Enseignement_Superieur_HITL_Audit_final.json (workflow HITL corrigé : routage Risque élevé / Risque faible / Else, sans clé API) et Agent_Cards_Governance_Package.json (générateur

Agent Card, configuration, runbook et commandes Git ; version d'exemple alignée sur le tag v1.0.0).

4. Agent Card

Champ	Valeur
Identifiant	EDU-ADM-001
Nom	EduAgent - Agent d'admission HITL
Objectif	Accélérer le tri des dossiers sans décider seul des admissions.
Propriétaire métier	Responsable pédagogique (commission d'admission)
Responsable technique	Équipe data et IA de l'établissement
Outils autorisés	Moteur de règles de scoring (actif); Modèle de langage (OpenRouter) (actif si OPENROUTER_API_KEY est défini, sinon simulation); Notification Slack d'escalade (simulé (aucun envoi réel)); Journal d'audit et d'exécution (actif)
Sources de données	Dossiers saisis via le formulaire du tableau de bord; Lot de test data/dossiers.csv (données fictives, sans e-mail)
KPI et seuils	succès min 80 % ; latence max 5000 ms ; échec max 20 % ; budget 2000 tokens/lot ; seuil HITL 70
Garde-fous	Kill switch : suspension immédiate de tout traitement (POST /api/admin/kill-switch); Rollback vers une version stable (POST /api/admin/rollback ou scripts/gouvernance.py rollback); Workflow d'approbation à deux rôles avant tout déploiement; Non-régression sur les cas golden avant déploiement; Dossier traité comme donnée non fiable : aucune instruction du dossier n'est exécutée; Score absent ou ambigu : escalade par défaut
Création / dernière revue / prochaine revue	2026-10-28 / 2026-10-28 / 2027-01-28

Limites : Ne prend aucune décision d'admission finale ; Les règles ne couvrent que les cinq critères du dossier de test ; Notification Slack simulée dans cette version ; Seuils à valider par la commission avant mise en production

5. Versionnement SemVer

Version	Statut	Origine
1.0.0	stable (active)	version initiale
1.1.0	retirée par rollback	CR-002 : seuil HITL 70 -> 90
1.0.1	en revue	CR-003 : latence d'alerte 5 s -> 4 s

Major : refonte ou rupture. Minor : changement de routage rétro-compatible. Patch : correction ou ajustement.

6. Résultats du lot de test

Indicateur	Valeur
Dossiers traités	14
Taux de succès	78.6 %
Taux d'échec	21.4 %
Latence moyenne	1678 ms
Tokens consommés	2397
Coût total	0.000828 USD

7. Simulation d'incident (runbook)

Régression de seuil HITL après déploiement approuvé (CR-002)

Étape	Résumé
0-deploiement	version 1.1.0 déployée (seuil HITL 70 -> 90)
1-detection	2 dossier(s) à risque élevé classé(s) à risque faible
2-contention	kill switch actif ; traitement suspendu : oui
3-remediation	rollback 1.1.0 -> 1.0.0 ; anomalies restantes : 0
4-retablissement	kill switch désactivé ; dossier de contrôle : Risque élevé

Anomalies détectées : 2 ; après rollback : 0 ; kill switch efficace : oui.

Enseignement : les cas golden doivent couvrir le score 70 (fraude seule) avant tout changement de seuil.

8. Sécurité

- Aucune clé dans le dépôt ; les clés du flow d'origine ont été révoquées.
- Minimisation des données : aucune donnée de contact transmise au modèle.

- Jeton administrateur obligatoire pour le kill switch et le rollback.
- Le contenu du dossier est traité comme donnée non fiable.

9. Déploiement et intégrations sécurisées

- Conteneurisation Docker et Docker Compose : environnement reproductible, API Agent (FastAPI) séparée de ses données, variables d'environnement centralisées (.env, non versionné).
- Hébergement Cloud possible sur Azure Container Apps (entreprise, auto-scaling), Render (GitHub, HTTPS automatique) ou Railway (prototypes et MVP). Aucun déploiement Cloud n'a été réalisé dans ce projet.
- Secrets : jamais de clé API dans les prompts ni dans le dépôt ; variables d'environnement sécurisées du serveur.
- Flux : HTTPS pour les appels entre un workflow externe et l'API, principe du moindre privilège pour les clés d'accès.

10. Tests et qualité

37 tests automatisés (scoring, gouvernance, simulation d'incident, API) : python -m pytest -q.

11. Limites et suites

- Notification Slack simulée
- Seuils à valider par la commission avant production
- Données fictives uniquement
- Tags Git non présents dans l'archive : à créer lors de la publication (git tag v1.0.0, puis v1.1.0 si l'historique de l'incident est conservé)